Compte Rendu — TP Android : Liste de Contacts (Version 2)

Étudiant : Diallo Mamadou Lamarana

Matière : Développement Mobile Android

Technologies : Kotlin, Android Studio, Interfaces XML, Intents explicites & implicites, FileProvider, Activity Result API.

---

1. Introduction & Objectifs du TP

Ce travail pratique consiste à développer une application Android native en Kotlin permettant de gérer une liste de contacts. L'application se compose de deux activités principales :

- **MainActivity** : affiche la liste personnalisée des contacts dans une `ListView` avec photo, nom, prénom et numéro de téléphone.
- **DetailActivity** : permet de consulter et de modifier les informations d'un contact sélectionné, de passer un appel téléphonique, d'envoyer un e-mail, ainsi que d'associer une photo prise via l'appareil photo ou choisie dans la galerie du téléphone.

2. Principales étapes de réalisation

A. Modèle de données (Contact.kt)

- Création d'une data class Kotlin `Contact` implémentant l'interface `Parcelable` afin d'être transmise efficacement entre activités via les `Intent`.
- Attributs du contact : `id`, `nom`, `prenom`, `telephone`, `email` et `photoUri`.

B. Écran principal et Adaptateur (MainActivity & ContactAdapter)

- **Layout item_contact.xml** : conception d'une carte personnalisée (`MaterialCardView`) comprenant une photo de profil circulaire (`ShapeableImageView`), le nom complet en gras, le téléphone et une icône flèche.
- **ContactAdapter.kt** : adaptateur héritant de `ArrayAdapter<Contact>` utilisant le pattern ViewHolder pour optimiser le recyclage des vues et l'affichage des photos (avec image par défaut `ic_person_placeholder` si aucune photo n'est définie).
- **MainActivity.kt** : initialisation d'une liste de contacts de test. Utilisation du contrat `registerForActivityResult(ActivityResultContracts.StartActivityForResult())` pour lancer `DetailActivity` et récupérer les données modifiées au retour.

C. Écran de détail et fonctionnalités (DetailActivity)

- **Layout activity_detail.xml** : interface d'édition complète respectant les maquettes demandées avec photo circulaire, badge appareil photo, champs de texte (`EditText`), boutons d'action rapide d'appel et d'e-mail, boutons photo et boutons de confirmation/annulation.
- **Intents implicites** : bouton « Appeler » déclenchant `Intent.ACTION_DIAL` avec le schéma `tel:` ; bouton « Envoyer e-mail » déclenchant `Intent.ACTION_SENDTO` avec le schéma `mailto:`.

- **Gestion d'erreurs** : blocs `try-catch (ActivityNotFoundException)` et affichage d'un `Toast` si aucune application d'appel ou de messagerie n'est installée.
- **Galerie** : utilisation de `ActivityResultContracts.GetContent()` pour sélectionner une image.
- **Appareil photo** : utilisation de `ActivityResultContracts.TakePicture()` avec configuration d'un `FileProvider` dans `AndroidManifest.xml` et `file_paths.xml` pour capturer des clichés en haute résolution.
- **Persistance des images** : les images sélectionnées ou capturées sont automatiquement copiées dans le dossier privé de l'application (`filesDir`) afin de conserver l'accès aux photos même après le redémarrage de l'application.
- **Validation & retour de résultat** : contrôle des champs obligatoires (Nom, Prénom, Téléphone) avec message Toast et erreurs sur les champs. Le bouton **Enregistrer** renvoie le Contact modifié avec `setResult(RESULT_OK)` et termine l'activité ; le bouton **Annuler** renvoie `setResult(RESULT_CANCELED)` sans enregistrer les modifications.

3. Tests effectués & Résultats

Test réalisé	Procédure de test	Résultat obtenu
Affichage de la liste	Lancement de l'application.	La ListView affiche correctement la liste des contacts préremplis.
Consultation & Édition	Clic sur un contact dans la liste.	DetailActivity s'ouvre avec toutes les informations pré-remplies.
Modification & Sauvegarde	Changement du nom et téléphone, clic sur « Enregistrer ».	Retour à MainActivity avec mise à jour immédiate de la liste.
Annulation	Modification d'un champ puis clic sur « Annuler » ou bouton retour.	Retour à la liste sans aucune modification enregistrée.
Appel téléphonique	Clic sur le bouton vert d'appel à côté du téléphone.	Ouverture du composeur téléphonique prérempli avec le numéro.
Envoi d'e-mail	Clic sur le bouton bleu à côté de l'e-mail.	Ouverture de l'application de messagerie avec l'adresse préremplie.
Sélection Galerie	Clic sur « Choisir une photo » et sélection d'un cliché.	La photo est immédiatement mise à jour et affichée dans le profil.
Prise de photo Caméra	Clic sur « Prendre une photo » et validation du cliché.	La photo prise est affichée et conservée dans l'application.
Validation champs vides	Effacement du champ Nom et clic sur « Enregistrer ».	Affichage d'un message d'erreur et blocage de l'enregistrement.

4. Difficultés rencontrées et solutions

1. Persistance et permissions d'accès aux photos de la galerie

- *Problème* : L'URI renvoyée par le sélecteur de galerie perd ses permissions d'accès en lecture après la fermeture de l'activité ou le redémarrage de l'application.

- *Solution* : Implémentation d'une fonction utilitaire `copyUriToInternalStorage()` qui copie le flux de données de l'image sélectionnée directement dans le stockage interne privé de l'application (`filesDir`).

2. Capture de photos HD avec l'appareil photo (FileProvider)

- *Problème* : À partir d'Android 7.0 (API 24), le passage d'URI `file://` brutes entre applications déclenche une `FileUriExposedException`.
- *Solution* : Configuration d'un `FileProvider` dans `AndroidManifest.xml` associant le fichier `res/xml/file_paths.xml` pour générer des URI sécurisées `content://`.

3. Visibilité des Intents implicites sous Android 11+ (API 30+)

- *Problème* : Les appels `ACTION_DIAL` ou `ACTION_SENDTO` peuvent échouer si les applications cibles ne sont pas déclarées.
- *Solution* : Ajout de la balise `<queries>` dans `AndroidManifest.xml` spécifiant les schémas `tel` et `mailto`.